



RS001 Week 12 — English Worksheet

Topic: Three Battles, Three Different Choices · **Course:** Text Adventure (Python) · **Time:** about 45 minutes

This week you wrap a whole battle into one **function** called `fight()`, then call it three times with different enemies and different action menus. One function, many battles — no copy-paste.

```
def fight(enemy_name, actions):
    game.say(f"=== {enemy_name} appears! ===")
    game.say(f"Choices: {' '.join(actions)}")
    action = game.ask("Do what? ").strip().lower()
    if action == actions[0]:
        game.say("A hard-fought win!")
        return True
    return False

fight("forest goblin", ["fight", "trap", "run"])
```

1 · Predict 🧠

Read each piece of code. Before you run it in your head, write what you think happens.

```
actions = ["fight", "trap", "run"]
game.say(", ".join(actions))
```

`join` glues the list together. What single line is printed?


```
def fight(enemy):
    game.say(f"{enemy} appears!")
    return True

won = fight("orc")
game.say(won)
```

Two things print. What are they?


```
def fight(enemy, actions):  
    game.say(f"{enemy}: {actions[0]}")  
  
fight("slime", ["fire", "spell"])  
fight("knight", ["sneak", "talk"])
```

The same function is called twice. What two lines appear on screen?

2 · Spot the Bug 🐛

Each block was meant to do something, but it is broken. Read what it is **supposed** to do, fix it, then explain why the original was wrong.

Bug A — `fight()` should tell the caller whether the player won by returning a value. Right now it prints but returns nothing, so `won` is empty.

```
def fight(enemy):  
    game.say(f"You beat the {enemy}!")  
  
won = fight("goblin")  
game.say(f"Victory: {won}")
```

Hint: add a `return True` at the end of the function.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug B — Each call should use a **different** action menu. Right now both battles share the exact same actions, even though they pass different lists.

```
def fight(enemy, actions):  
    game.say(f"{enemy}: {' '.join(['fight', 'run'])}")  
  
fight("slime", ["fire", "spell", "run"])  
fight("knight", ["sneak", "talk", "run"])
```

Hint: the function should use its `actions` parameter, not a hard-coded list.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug C — `', '.join(actions)` needs a **list** of words to glue. Here a single string was passed, so the menu prints letter by letter.

```
def fight(enemy, actions):  
    game.say(f"{enemy}: {'', '.join(actions)}")  
  
fight("bat", "fly")
```

Hint: wrap the action(s) in a list: `["fly"]`.

Write the fixed code:

Why was it wrong? Why does your fix work?

3 • 🎯 Three Battles

Open your game. Write a `fight()` function, then call it for **three** different enemies, each with its own action menu and its own result. Reuse the function — do not copy-paste the battle code three times.

When it works, send a **photo or video**, then explain what you did. Use these sentence starters — write 4 to 6 sentences total.

First, I wrote a `fight()` function that ...

My three enemies were ...

Each enemy had a different menu, for example ...

Reusing the function meant I only had to fix bugs ...

One tricky moment was when ...

If I had more time, I would ...

4 · Record Your Walkthrough 🎥

Take a video on your phone (or a parent's phone) while your three battles run. Teach it like the viewer has never coded. Try to use these words: **function**, **call**, **parameter**, **return**, **reuse**.

1. Run your game through all three battles.
2. Point to the one `fight()` function in your code.
3. Show the three different calls and their different menus.
4. Explain why one function is better than three copies.

Write what you will say in your video. Plan it here first.

Submit ✓

Send this worksheet + your walkthrough video to teacher on KakaoTalk.